POLICY MANAGEMENT SERVICE

University of Southampton

UNIVERSITY OF Southampton

TABLE OF CONTENTS

Introduction 1

Section 1 - The Policy Management Interface 2

Chapter One: The Policy Tab 3

Chapter Two: The Ontology Tab 5

Section 2 - The Policy Editor Interface 7

Chapter Four: Using the Default Editor 8

Section 3 - The Policy Editor and the Ontology Engine 14

Chapter Five - Custom Ontology Creation Guidance 14

Chapter Six – Policy Specification 25

Chapter Seven: Adding the Ontology 32

Section 4 – The Policy Engine 36

Chapter Eight: Policy Evaluation 37

Introduction

This document introduces usage of the UoS SODRL Policy Service.

It is written to guide the user through usage of the user interface and the underlying key functionalities.

The key functionalities currently supported include:

- The SODRL Policy Editor
    - Policy creation and maintenance
- The SODRL Policy Engine
    - Policy Evaluation
    - Policy Comparison
- The SODRL Ontology Service
    - Ontology maintenance

Section 1 - The Policy Management Interface

This section provides an overview of the main Policy Management page which provides the user with centralized access to all the key functionalities.

The user interface with example content for illustration is shown in Figure 1 and Figure 2

The Policy Tab that provides access to policy related functionality. This is described in more detail in Chapter One: The Policy Tab

Figure 1 - Policy Management Interface: Policy Tab

The Ontology Tab that provides access to supporting ontology related functionality. This is described in more detail in Chapter Two: The Ontology Tab

Figure 2 - Policy Management Interface: Ontology Tab

Chapter One: The Policy Tab

This chapter provides a summary of the Policy Tab on the main management page, as illustrated in Figure 1.

Overview

This page will display all current policies created by the user. In the example shown the user has created just one policy named “DATAPACTDEMO_EXAMPLE_POLICY”.

For each one of the policies the user is offered four management options:

- If the user clicks on the policy name, then the user will be taken to the policy edit interface. This is described in more detail below.
- If the user clicks on the ‘download’ button (green) then the policy will be downloaded to the user’s machine in json-ld format.
- By default, the policies created by the users are marked as private and not visible to others. The user may toggle between private and public visibility by clicking on the ‘publish’ / ‘protect’ button (blue). An example is shown in Figure 3 and Figure 4

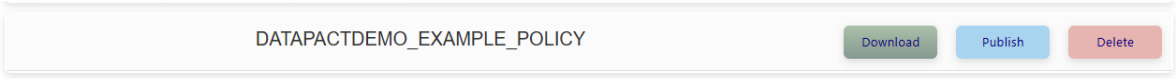

Figure 3 - Private Policy

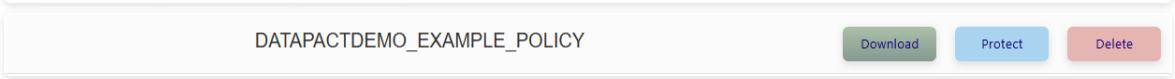

Figure 4 - Public Policy

- If the user clicks on the ‘delete’ button (red) then the policy will be permanently deleted from within this system.

The user is also offered three more general options. These are shown above the list of policies and illustrated in Figure 5.

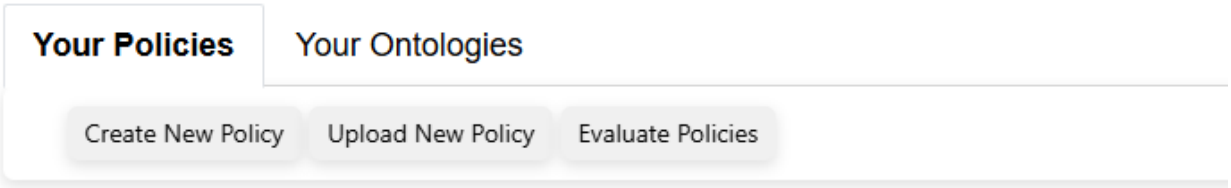

Figure 5 - General policy page options

- If the user clicks on the ‘Create New Policy’, then the user will be taken to the policy edit interface. This is described in more detail below.
- If the user clicks on the ‘Upload Policy’, then the user may upload a previously saved json-ld format policy and will be taken to the policy edit interface. This is described in more detail below.
- If the user clicks on the ‘Evaluate Policies’, then the user will be taken to the policy evaluation interface. This is described in more detail below.

Chapter Summary/Key Takeaways

This tab provides navigation of the high-level functionality available for policy management.

In summary the user may:

- Create/View/Update/Delete policies.
- Upload/Download policies.
- Evaluate policies.

Chapter Two: The Ontology Tab

This chapter provides a summary of the Ontology Tab on the main management page, as illustrated in Figure 2.

Overview

This page will display all ontologies created by the user, plus the default ontology provided with the service. In the example shown the user has created no ontologies and only the default ontology “ODRL_DPV.rdf” is shown.

In this context, the ontologies are used primarily to provide vocabulary used to populate the dropdowns used by the policy editor to define policies. Domain specific ontologies may be created to define domain specific vocabulary that may be required to define appropriate policies.

The default ontology provides a base vocabulary that is used in the dropdowns. It has been created by combining and aligning existing vocabulary across ODRL (Open Digital Rights Language) and DPV (Data Privacy Vocabulary).

For each one of the ontologies the user is offered two management options:

- By default, the ontologies created by the users are marked as private and not visible to others. The user may toggle between private and public visibility by clicking on the ‘publish’ / ‘protect’ button (blue). An example is shown in Figure 6 and Figure 7.

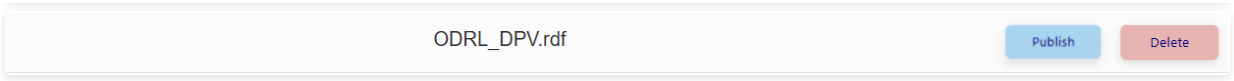

Figure 6 - Private Ontology

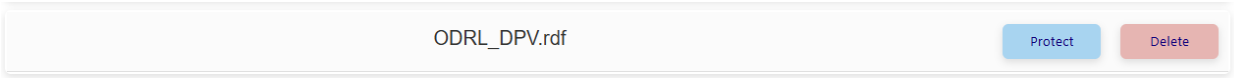

Figure 7 - Public Ontology

- If the user clicks on the ‘delete’ button (red) then the policy will be permanently deleted from within this system.

The user is also offered one more general option. This is shown above the list of ontologies and illustrated in Figure 8.

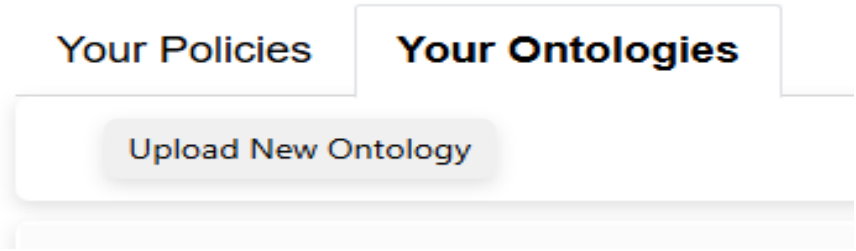

Figure 8 - General ontology page options

- If the user clicks on the 'Upload Ontology', then the user may upload a previously saved turtle or RDF format ontology file.

Chapter Summary/Key Takeaways

This tab provides navigation of the high-level functionality available for ontology management.

In summary the user may:

- Upload/Delete ontologies.

Section 2 - The Policy Editor Interface

This section provides an overview of the Policy Editor which provides the user with an easy-to-use method of creating ODRL policies.

The user interface with example content for illustration is shown in Figure 9. In this example the policy has been created with a single rule to describe usage for the data asset "http://example.org/datasets/climateChangeData".

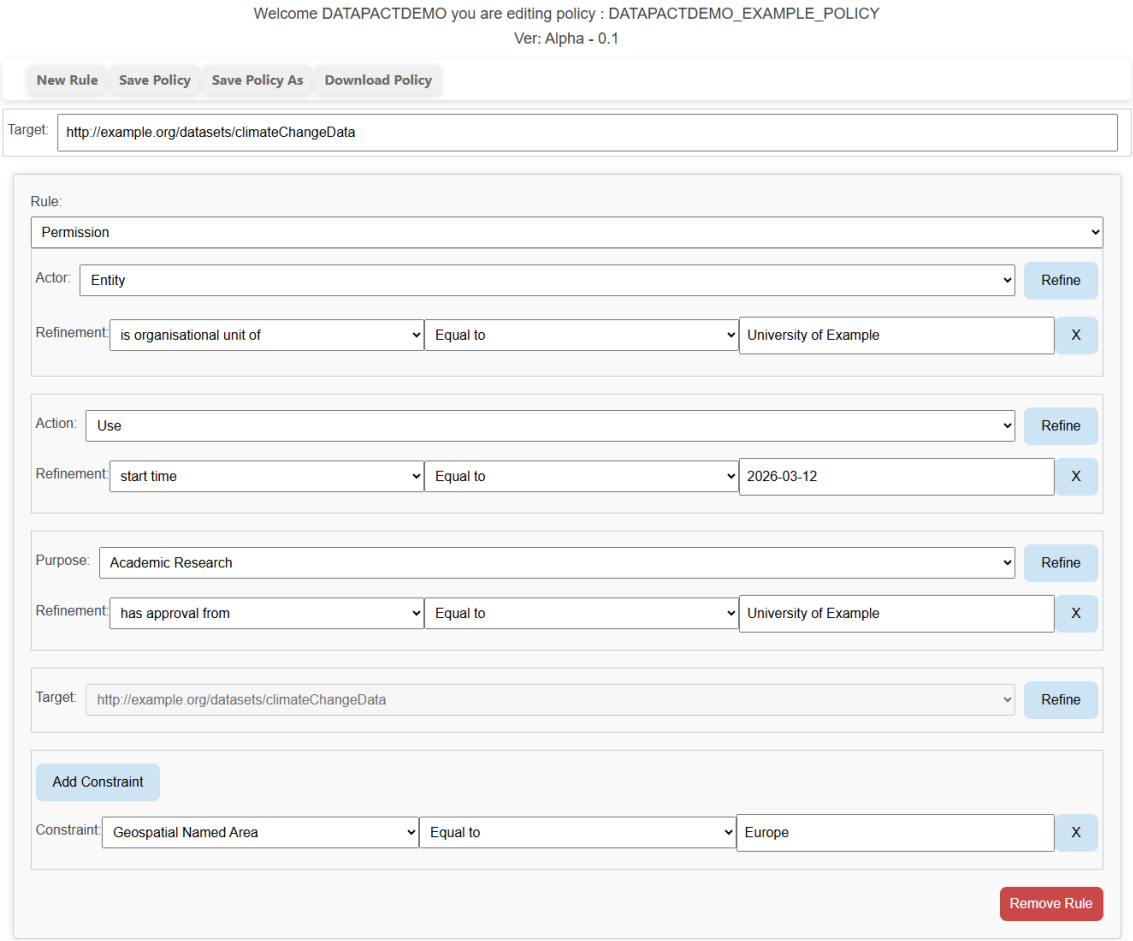

Figure 9 - The Policy Editor Interface

Chapter Four: Using the Default Editor

This chapter provides a summary of how to use the default editor configuration.

Introduction

ODRL policies are created to describe details of how a data asset may be used in a machine-readable form.

Currently each policy describes the details for a single asset. This is defined in the Target parameter within the UI as shown in Figure 10

Figure 10 - Target Parameter

The following text will describe the structure of the User Interface and how a policy is created.

Rules

A policy is created against a data asset by describing one or more rules.

A new rule is created by pressing the New Rule button at the top of the screen, this is illustrated in Figure 11.

Figure 11 - New Rule

This will add a new empty rule to the policy; this is illustrated in Figure 12. This or any existing rules may be removed using the 'Remove Rule' button seen in the bottom right.

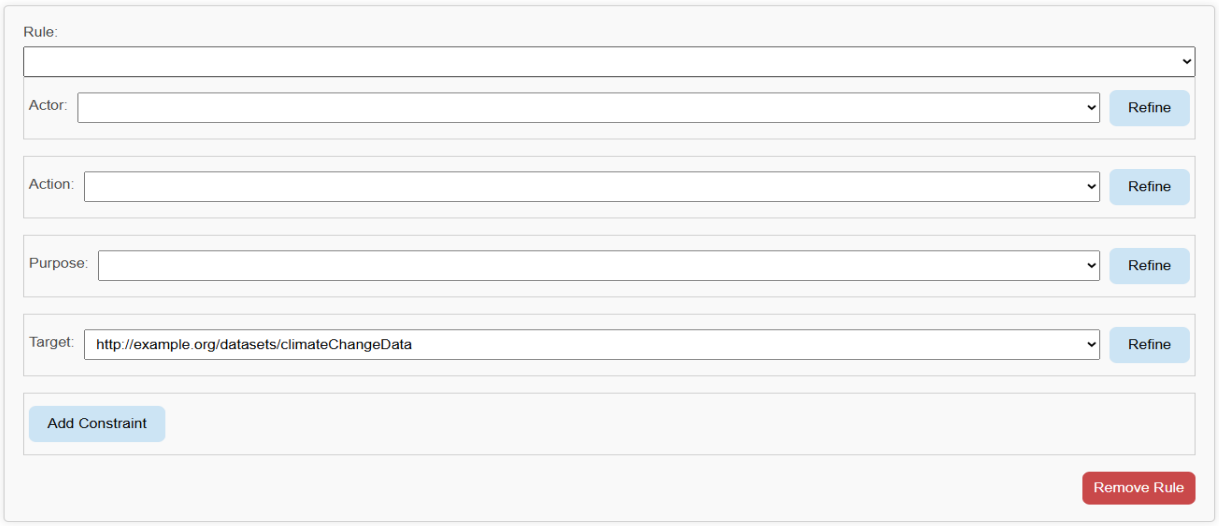

Figure 12 - New Rule Template

A user may then define one of three different rule types by selecting form the Rule dropdown menu. This is illustrated in Figure 13.

The user should select one of:

- Permission: The elements of the rule are permitted.
- Prohibition: The elements of the rule are prohibited.
- Obligation: An additional obligation required of the user.
- **NOTE:** Duty should not be used, it is currently maintained for future extensions but will be removed

Figure 13 - Rule Dropdown

A rule is constructed from the following elements (these will be described in a bit more detail below)

- Actor – Who this rule applies to.
- Action – What the Actor may/may not/is obliged to do.
- Purpose – Why the Actor is executing the action.

- Target – This is the same target as discussed above and in this context just allows further refinement.
- Constraint – Constraint applying to all rule elements.

Actors

Within a rule the Actor is defined by selection from the dropdown list. The items in the drop-down menu are derived from the underlying ontology. This is illustrated in Figure 14.

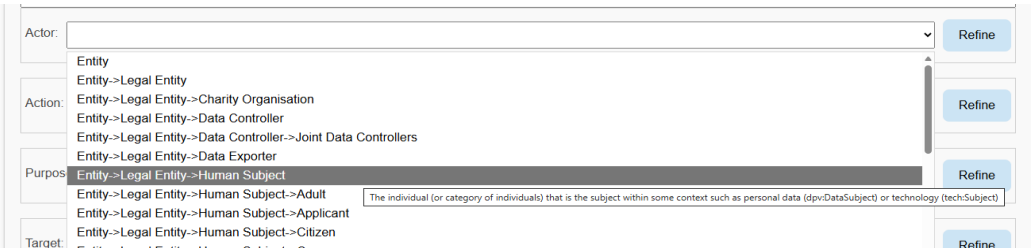

Figure 14 - Actor selection

Action

Within a rule the Action is defined by selection from the dropdown list. The items in the drop-down menu are derived from the underlying ontology. This is illustrated in Figure 15.

Figure 15 - Action Selection

Purpose

Within a rule the Purpose is defined by selection from the dropdown list. The items in the drop-down menu are derived from the underlying ontology. This is illustrated in Figure 15.

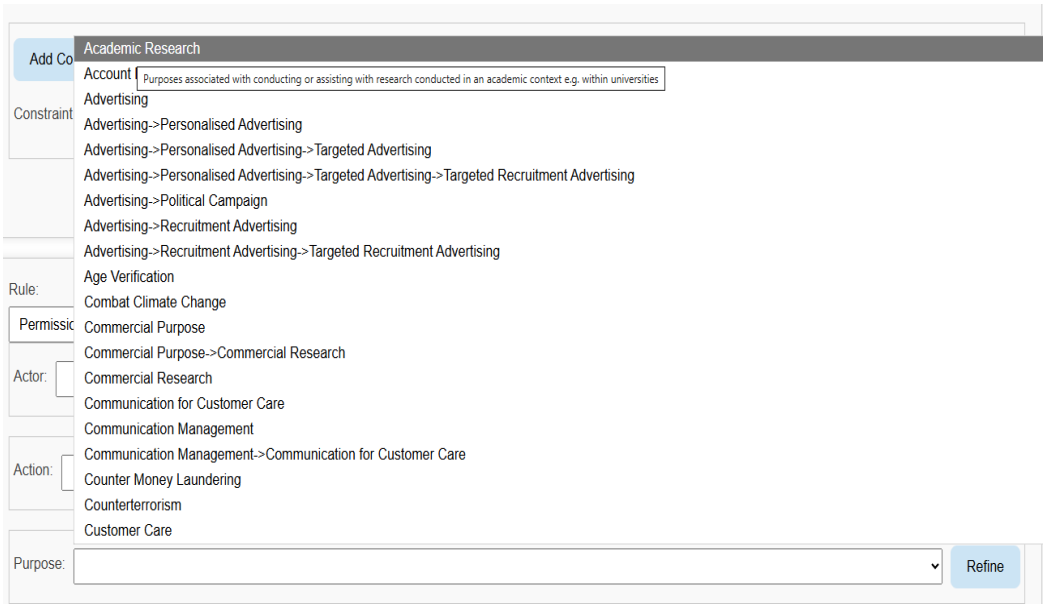

Figure 16 - Purpose Selection

Target

Within a rule the Target cannot be changed. It will be set at the value of the target for the policy (as described above). However, refinements may be specified for the Target to limit the scope of the rule coverage in the data. How a user may add 'refinements' is described further below.

Refine

Each of the rule elements (Actor, Action, Purpose, Target) can also be further refined to provide more detailed qualification of these elements. An example is shown in Figure 17.

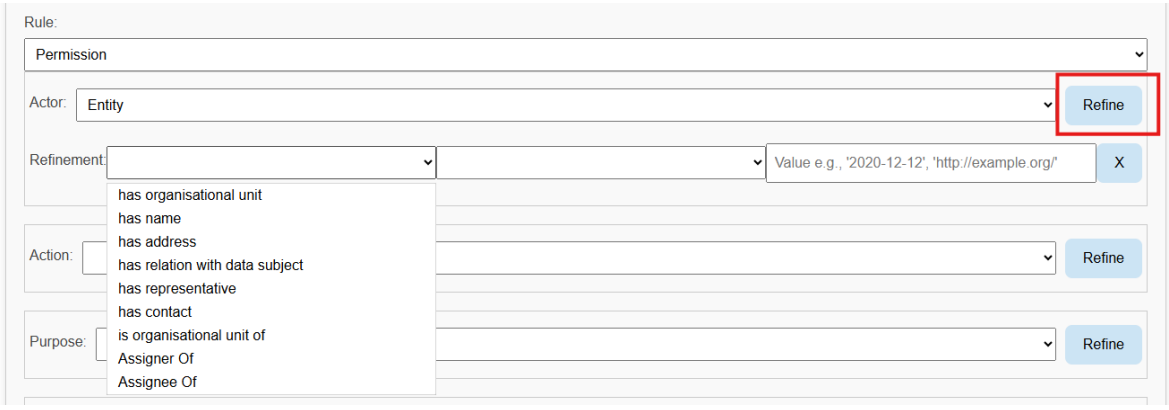

Figure 17 - Refinement example

In this example (which is generally true for any of the elements) the user has pressed the Refine button (highlighted) to add a refinement dialogue for this element. This dialogue allows for the

creation of expressions which compare two operands in order to further qualify (constrain) the element (if the expression is true the refinement is satisfied and if false it is not satisfied).

The items in the left operand and operator (middle field) are selected from a drop-down menu. The items in the drop-down menu are context sensitive (i.e. in this case depend on the Actor item selected) and are derived from the underlying ontology. The items in the right operand are free text.

The user may add as many refinements as required. The user may delete refinements using the 'cross' button shown on the right-hand side of the refinement.

Constraint

The whole rule may also be further constrained to provide more detailed qualification of the whole rule. An example is shown in Figure 18.

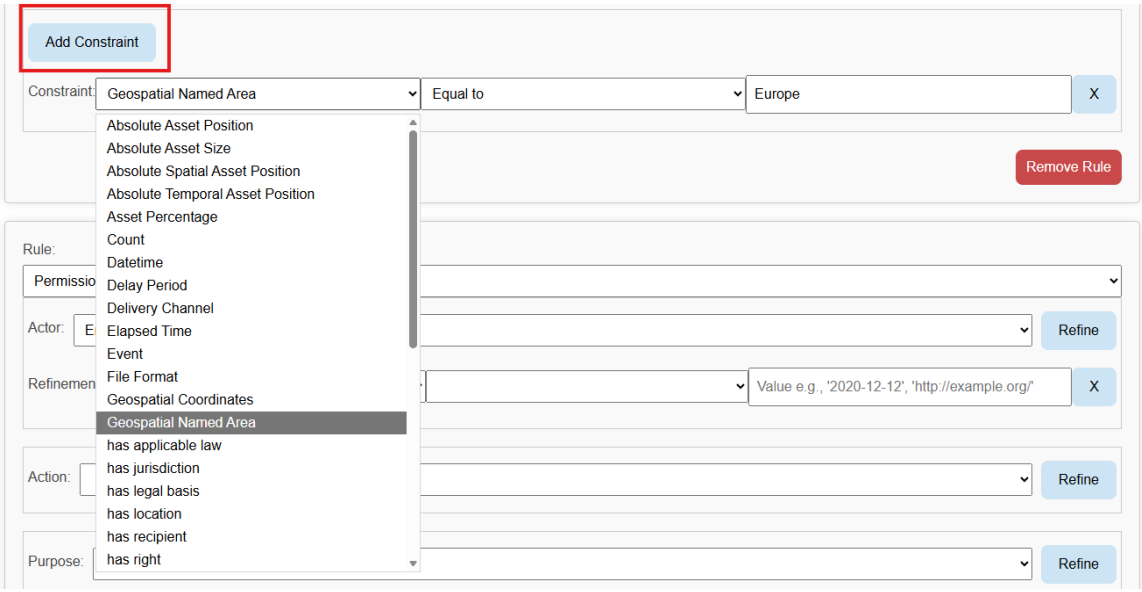

Figure 18 – Constraint Example

In this example (which is generally true for any of the elements) the user has pressed the Add Constraint button (highlighted) to add a constraint dialogue for this element. This dialogue allows for the creation of expressions which compare two operands in order to further qualify (constrain) the rule (if the expression is true the refinement is satisfied and if false it is not satisfied).

The items in the left operand and operator (middle field) are selected from a drop-down menu. The items in the drop-down menu are derived from the underlying ontology. The items in the right operand are free text.

The user may add as many constraints as required. The user may delete constraints using the ‘cross’ button shown on the right-hand side of the constraint.

Chapter Summary/Key Takeaways

The default editor configuration allows a user the flexibility to create policies using the default vocabulary based on a refined amalgam of the ODRL and DPV vocabularies.

Policies for a single data asset are created from one or more rules. The rules may define Permissions, Prohibitions or Obligations. Each rule is targeted by defining the relevant Actor, Action and Purpose to which it applies. Refinement and constraints may also be provided to further define the specificity of the rule targeting.

Section 3 - The Policy Editor and the Ontology Engine

As described above, the vocabulary used by the editor is driven by the underlying ontologies being used. In addition to the default ontology provided with the editor the user may also add domain specific ontologies to refine expression of the required policies.

This section will show how ontologies may be added and how they affect the vocabulary within the editor.

Chapter Five - Custom Ontology Creation Guidance

This chapter provides specifics on how a custom ontology may be created to be used by the editor.

Introduction

The ODRL Editor is supplied with a default underpinning ontology (an integration of ODRL and DPV). This is used to drive the vocabulary made available to the user when creating new policies.

The ODRL editor also provides the user with the ability to add domain specific ontologies that may be used to tailor the expressivity of the vocabulary to express specific business requirements.

The editor will allow the user to load custom ontologies using either Turtle (ttl) or RDF/XML (xml) files.

The text below provides some detail on how the editor queries the underpinning ontologies to extract to relevant vocabulary. This is then used to provide insight into how custom ontologies can be created that will tailor the editor's behaviour.

Some simple examples are also given that can be used as a basis for simple definitions.

Ontology creation should follow basic analysis of the business process(es) and related vocabulary that subsequent policies will serve.

Adding a New Action

Action Discovery

The editor will retrieve the list of possible Actions from the base ontology and additional custom ontologies using the following SPARQL query

```
SELECT DISTINCT ?action ?label  ?sub_action ?sub_label
      WHERE {

         ?sub_action   (rdf:type /  rdfs:subClassOf?)   odrl:Action .
         ?sub_action ( rdfs:label | skos:prefLabel ) ?sub_label .

         ?sub_action   odrl:includedIn | rdfs:subClassOf   ?action .
         ?action    ( rdfs:label | skos:prefLabel )    ?label .

      }
"""
```

From this query a hierarchical list of possible Actions is provided to the editor user -interface and this will be formatted and displayed in the Rule dropdown elements.

Custom Actions

An example of adding an Action is shown below (this is presented through a TTL fragment)

```
:CorrelationAnalysis a odrl:Action , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "Correlation Analysis"@en ;
        odrl:includedIn dpv-owl:Analyse .
```

Some commentary on this is given below

In this example the user has declared *CorrelationAnalysis* as *rdf:type odrl:Action*

The user may well have declared it as *rdfs:SubClassOf odrl:Action* or indeed *rdfs:SubClassOf* another user defined *rdf:type odrl:Action*. This would depend on the user's business modelling requirements. Any of these formulations would be discovered by the editor.

CorrelationAnalysis is also includedIn *dpv-owl:Analyse. F*or more specific detail on odrl:includedIn see ODRL Vocabulary & Expression 2.2 however broadly this is creating usable ancestor relationships. That is "An Action (except for use and transfer) MUST have one

includedIn property value (of type Action) to transitively assert this Action that encompasses its operational semantics." (see Ref 2.4)

In summary the user has the following options for a new action:

- The user adds a triple {new_action} rdf:type odrl:Action.
- The user defines their own class of actions {user_defined_actions}, add the triple {user_defined_actions} rdfs:subClassOf odrl:Action, and the triple {new_action} rdf:type {user_defined_actions}.
- The user adds a triple {new_action} rdfs:subClassOf {another_action} where {another_action} is of type odrl:Action.

And also

- Same as above but odrl:includedIn instead of rdfs:subClassOf.

Action Property Discovery

For a given selected Action the editor will populate a list of possible left operands to be used in associated Refinements (if any).

```
SELECT DISTINCT ?property ?label
      WHERE {
        {class_uri} rdf:type?   /  ( odrl:includedIn? | skos:broader? | rdfs:subClassOf* )
?class .
        ?property ( rdfs:domain | schema:domainIncludes | dcam:domainIncludes ) /
                                                ( skos:broader | rdfs:subClassOf)? ?class ;
                ( rdfs:label | skos:prefLabel ) ?label .
        FILTER ( ?property NOT IN ( odrl:includedIn, odrl:implies, odrl:hasPolicy) )
      }
 """
```

From this query a list of possible properties is provided to the editor user interface and this will be formatted and displayed in the left operand dropdown of the refinement dialogue

Custom Action Properties

An example of adding an Action with a custom action property is shown below (this is presented through a TTL fragment)

```
:GridBasedClustering a odrl:Action , skos:Concept , dpv-owl:Processing ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "Grid-based Clustering"@en ;
        odrl:includedIn dpv-owl:Analyse .
:hasThresholdDensity rdfs:domain :GridBasedClustering.
```

Also for completeness

```
:hasThresholdDensity a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "has threshold density"@en .
```

Some commentary on this is given below

In this example the user has declared *:hasThresholdDensity rdfs:domain :GridBasedClustering.* (alongside the Action *GridBasedClustering*).

By placing this in the *rdfs:domain* of *GridBasedClustering* then *hasThresholdDensity* is found through the query above and will be displayed as an option in the refinement dialogue.

The query provides some additional flexibility in tailoring the precise property semantics (i.e. *(rdfs:domain | schema:domainIncludes | dcam:domainIncludes) (skos:broader | rdfs:subClassOf)?*) however the example given provides a simple direct method of declaration. *skos:broader* and it's inverse *skos:narrower*, provides additional approaches to clarifying hierarchical semantic relationships (see SKOS Simple Knowledge Organization System Namespace Document - HTML Variant, 18 August 2009 Recommendation Edition).

For ODRL the refinements for Actions are considered as type *odrl:constraint* (itself having a structure of { *odrl:leftOperand, odrl:Operator*, *odrl:rightOperand*}. For this reason, the typing of *hasThresholdDensity* is included above.

Adding a New Actor

Actor Discovery

The editor will retrieve the list of possible Actors from the base ontology and additional custom ontologies using the following SPARQL query

```
SELECT DISTINCT ?actor ?label ?sub_actor ?sub_label

    WHERE {
      ?actor rdfs:subClassOf* dvpowl:Entity .
      ?actor skos:prefLabel|rdfs:label ?label .

      ?sub_actor rdfs:subClassOf ?actor .
      ?sub_actor skos:prefLabel|rdfs:label ?sub_label .

    }
"""
```

From this query a hierarchical list of possible Actors is provided to the editor and this will be formatted and displayed in the Rule dropdown elements.

Custom Actors

An example of adding an Actor is shown below (this is presented through a TTL fragment)

```
:PartTimeEmployee a rdfs:Class , skos:Concept ;
        rdfs:subClassOf dpv-owl:Employee ;
        rdfs:label "part-time employee"@en ;
        skos:broader dpv-owl:Employee .
```

Some commentary on this is given below

In this example the user has declared *PartTimeEmployee* as *rdf:type rdfs:Class*

Further they declared *PartTimeEmployee* as *rdfs:subClassOf dpv-owl:Employee*

In this case this has also declared *PartTimeEmployee* as *skos:broader dpv-owl:Employee* (for more specific detail on the semantic hierarchical relation *skos:broader* (see SKOS Simple Knowledge Organization System Namespace Document - HTML Variant, 18 August 2009 Recommendation Edition)

The *rdfs:subClassOf dpv-owl:Employee* formulation would be discovered by the editor (as *dpv-owl:Employee* is itself a descendant of *dvpowl:Entity* via *Employee->HumanSubject->LegalEntity->Entity*).

Actor Property Discovery

For a given selected Actor the editor will populate a list of possible left operands to be used in associated Refinements (if any).

```
SELECT DISTINCT ?property ?label
      WHERE {
        {class_uri} rdf:type?   /  ( odrl:includedIn? | skos:broader? | rdfs:subClassOf* )
?class .
        ?property ( rdfs:domain | schema:domainIncludes | dcam:domainIncludes ) /
                                                ( skos:broader | rdfs:subClassOf)? ?class ;
                ( rdfs:label | skos:prefLabel ) ?label .
        FILTER ( ?property NOT IN ( odrl:includedIn, odrl:implies, odrl:hasPolicy) )
      }
 """
```

From this query a list of possible properties is provided to the editor and this will be formatted and displayed in the left operand dropdown of the refinement dialogue

Custom Actor Properties

An example of adding an Actor is with a custom property is shown below (this is presented through a TTL fragment)

```
:ContractEmployee a rdfs:Class , skos:Concept ;
        rdfs:subClassOf dpv-owl:Employee ;
        rdfs:label "part-time employee"@en ;
        skos:broader dpv-owl:Employee .
:hasWorkRights rdfs:domain :ContractEmployee .
```

Also for completeness

```
: hasWorkRights a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "has work rights"@en .
```

Some commentary on this is given below

In this example the user has declared *:hasWorkRights rdfs:domain :ContractEmployee.* (alongside the Actor *ContractEmployee*).

By placing this in the *rdfs:domain* of *ContractEmployee* then *hasWorkRights* is found through the query above and will be displayed as an option in the refinement dialogue.

The query provides some additional flexibility in tailoring the precise property semantics (i.e. *(rdfs:domain | schema:domainIncludes | dcam:domainIncludes)*

(skos:broader | rdfs:subClassOf)?) however the example given provides a simple direct method of declaration.

For ODRL the refinements for Actors are considered as type *odrl:constraint* (itself having a structure of {leftOperand, Operator, rightOperand}. For this reason the typing of *hasWorkRights* is included above.

Adding a New Purpose

Purpose Discovery

The editor will retrieve the list of possible Purposes from the base ontology and additional custom ontologies using the following SPARQL query

```
SELECT DISTINCT ?purpose ?label  ?sub_purpose ?sub_label
      WHERE {

          ?sub_purpose rdf:type dvpowl:Purpose .
          ?sub_purpose ( rdfs:label | skos:prefLabel ) ?sub_label.

          ?sub_purpose ( skos:broader | rdfs:subClassOf) ?purpose .
          ?purpose ( rdfs:label | skos:prefLabel ) ?label .


        }
    """
```

From this query a a hierarchical list of possible Purposes is provided to the editor and this will be formatted and displayed in the Rule dropdown elements.

Custom Purposes

An example of adding a Purpose is shown below (this is presented through a TTL fragment)

```
:MedicalResearch a dpv-owl:Purpose , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "Medical Research"@en ;
        skos:prefLabel "Medical Research"@en ;
        skos:broader dpv-owl:ScientificResearch .
```

Some commentary on this is given below

In this example the user has declared *MedicalResearch* as *rdf:type dpv-owl:Purpose* and so is discovered by the editor.

In this case this has also declared *MedicalResearch* as *skos:broader dpv-owl:Employee* (for more specific detail on the semantic hierarchical relation *skos:broader* see SKOS Simple Knowledge Organization System Namespace Document - HTML Variant, 18 August 2009 Recommendation Edition)

Purpose Property Discovery

For a given selected Purpose the editor will populate a list of possible left operands to be used in associated Refinements (if any).

```
SELECT DISTINCT ?property ?label
      WHERE {
        {class_uri} rdf:type?   /  ( odrl:includedIn? | skos:broader? | rdfs:subClassOf* )
?class .
        ?property ( rdfs:domain | schema:domainIncludes | dcam:domainIncludes ) /
                                                ( skos:broader | rdfs:subClassOf)? ?class ;
                ( rdfs:label | skos:prefLabel ) ?label .
        FILTER ( ?property NOT IN ( odrl:includedIn, odrl:implies, odrl:hasPolicy) )
      }
 """
```

From this query a list of possible properties is provided to the editor and this will be formatted and displayed in the left operand dropdown of the refinement dialogue

Custom Purpose Properties

An example of adding an Purpose is with a custom property is shown below (this is presented through a TTL fragment)

```
:MedicalResearch a dpv-owl:Purpose , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "Medical Research"@en ;
        skos:prefLabel "Medical Research"@en ;
        skos:broader dpv-owl:ScientificResearch .
:hasSponsor rdfs:domain :MedicalResearch .
```

Also for completeness

```
: hasSponsor a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "is sponsored by"@en .
```

Some commentary on this is given below

In this example the user has declared *:hasSponsor rdfs:domain :MedicalResearch.* (alongside the Purpose *MedicalResearch*).

By placing this in the *rdfs:domain* of *MedicalResearch* then *hasSponsor* is found through the query above and will be displayed as an option in the refinement dialogue.

The query provides some additional flexibility in tailoring the precise property semantics (i.e. *(rdfs:domain | schema:domainIncludes | dcam:domainIncludes) (skos:broader | rdfs:subClassOf)?*) however the example given provides a simple direct method of declaration.

For ODRL the refinements for Purposes are considered as type *odrl:constraint* (itself having a structure of {leftOperand, Operator, rightOperand}. For this reason the typing of *hasSponsor* is included above.

Adding a New Constraint

Constraint Left Operand Discovery

The editor will retrieve the list of possible left operands for Constraints from the base ontology and additional custom ontologies using the following SPARQL query

```
SELECT DISTINCT ?leftoperand ?label
        WHERE {
          ?leftoperand rdf:type odrl:LeftOperand ;
                ( rdfs:label | skos:prefLabel ) ?label .
        }
    """
```

From this query a hierarchical list of possible left operands for Constraints is provided to the editor and this will be formatted and displayed in the Rule dropdown elements.

Custom Constraint Left Operands

An example of adding a left operand for a constraint is shown below (this is presented through a TTL fragment)

```
:hasThresholdDensity a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
        rdfs:isDefinedBy <http://example.org> ;
        rdfs:label "has threshold density"@en .
```

Some commentary on this is given below

In this example the user has declared *hasThresholdDensity* as *rdf:type hasThresholdDensity* and so is discovered by the editor.

Chapter Summary/Key Takeaways

This chapter provides quite technical and detailed information to support user's creation of domain specific ontologies that can be loaded into the editor.

By following these directions ontologies may be created and loaded into the editor to tailor the vocabulary available to the user when specifying policies.

Chapter Six – Policy Specification

This chapter provides specifics on how JSON-LD ODRL policies are output by the policy editor.

Introduction

This section should be read in conjunction with ODRL Information Model 2.2 and the ODRL Editor User Interface instructions.

It is intended to provide guidance in using the ODRL fragment developed by the University of Southampton within the EU UPCAST project. It is intended to further clarify how JSON-LD policy 'documents' will be created from the knowledge recorded in the editor user interface.

Detailed data references are described through JSON-LD fragments

Policy creation should follow basic analysis of the business process(es) and related vocabulary/grammar that subsequent policies will serve.

The Target

A policy is associated with a ***single*** Target.

This is used to identify a specific data asset.

The Rules (see below) are used to define the Policy

Rules

The basic building block of a policy is the Rule.

There may be one or more Rules that taken together are used to define the Policy.

There are three Rule types:

- Permission
- Prohibition
- Obligation

The Rule Type 'Duty' is currently also displayed in the user interface (for future development) but this should not be used.

The effect of multiple Rules is cumulative (that is the effects are AND ed).

Each Rule is made up of the following elements

- Actor (see ref 2.3)
  - Defines who this rule applies to .
  - See "odrl:Assignee" and "@type" : "odrl:PartyCollection" .
  - The value is defined directly or in element "odrl:source" : *URI* when of "@type" : "odrl:PartyCollection".
  - There may be none, one or many "odrl:refinement" (of type "odrl:constraint" - see Constraint below).
- Action (see ref 2.4)
  - Defines what action (on the Target) this rule applies to
  - See "odrl:action".
  - The value is defined directly or in element "rdf:value" : *IRI* (if refinements defined for the action – see below).
  - There may be none, one or many "odrl:refinement" (of type "odrl:constraint" - see Constraint below).
  -
- Purpose
  - Defines why the action (on the Target) is being applied.

  - See “odrl:purpose” (https://www.w3.org/TR/odrl-vocab 4.5.19)
    - This is of class “odrl:leftOperand”
    - The value is defined directly “odrl:leftOperand”: *IRI*
  - There may be none, one or many “odrl:refinement” (of type “odrl:constraint” - see Constraint below)
  - Purpose descriptions and refinements (if any) are presented as Constraints. This is best illustrated in the JSON-LD example given below, where the Purpose ‘block’ can be seen as part of the set of constraints (where Purpose refinements are presented these are tied with the Purpose using “odrl:and” (https://www.w3.org/TR/odrl-vocab 3.17.3)
- Target
  - This is used to identify a specific data asset.
  - See “odrl:target” and “@type” : “odrl:AssetCollection” .
  - The value is defined directly or in element “odrl:source” : *IRI* (if refinements defined for the target – see below).
  - Each rule allows the user to add none, one or many “odrl:refinement” (of type “odrl:constraint” - see Constraint below) (to the fixed target) that apply to that rule.
  - The odrl:leftoperand “QUERY” may be used to specify a subset of policy Target.
- Constraints (see ref 2.5)
  - Defines constraints applied to the Rule as a whole.
  - See “odrl:constraint”
    - This has:
      - “odrl:leftOperand”: *IRI*
      - “odrl:operator”: *IRI*
      - “odrl: rightOperand”: *IRI.*
  - There may be none, one or many “odrl:constraint” .

A Simple Example Policy Represented in JSON-LD

```
 "@context": {
     "dash": "http://datashapes.org/dash#",
     "rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
     "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
     "schema": "http://schema.org/",
     "sh": "http://www.w3.org/ns/shacl#",
     "xsd": "http://www.w3.org/2001/XMLSchema#",
     "ex": "http://example.ex/",
     "odrl": "http://www.w3.org/ns/odrl/2/"
   },
"@id": "https://eu-upcast.github.io/shapes/mint#2policy17446325067569091",
"@type": "odrl:Policy",
“odrl:permission”:
 [
   {
     “odrl:action”
     {
        “rdf:value” :”https://w3id.org/dpv/owl#Analyse",
        “odrl:refinement”:
        [
          {
             “odrl:leftOperand”: “https://eu-upcast.github.io/vocabulary/index-en.html#hasStartTime” ,
             “odrl:operator”: “http://www.w3.org/ns/odrl/2/eq”,
             “odrl:rightOperand”: “Today”
          },
          {
             “odrl:leftOperand”: “https://eu-upcast.github.io/vocabulary/index-en.html#hasFinishTime”,
             “odrl:operator:http”: “//www.w3.org/ns/odrl/2/eq”,
             “odrl:rightOperand”: “Tomorrow”
          }
        ]
     }
     “odrl:assignee”:
     {
        “@type” : “odrl:PartyCollection”,
        “odrl:source” : “https://w3id.org/dpv/owl#Student”,
        odrl:refinement:
        [
          {
             “odrl:leftOperand”: “https://w3id.org/dpv/owl#isRepresentativeFor”,
             “odrl:operator”:http: “//www.w3.org/ns/odrl/2/eq”,
             “odrl:rightOperand”: “University of Harvard”
          }
        ]
     }
     “odrl:target”;
     {
        “@type” : “odrl:AssetCollection”,
        “odrl:source” : “http://example.org/datasets/covid19Stats”,
        “odrl:refinement”:
        [
          {
             “odrl:leftOperand”: “QUERY”,
             “odrl:operator”: “http://www.w3.org/ns/odrl/2/eq”,
```

```
                "odrl:rightOperand": "SELECT C1, C2 FROM COVID_DATA"
              }
            ]
        }
        "odrl:constraint"
        [
          "odrl:and":
          [
            {
              "odrl:leftOperand": "odrl:purpose",
              "odrl:operator":http: "//www.w3.org/ns/odrl/2/eq",
              "odrl:rightOperand": "https://w3id.org/dpv/owl#AcademicResearch"
            },
            {
              "odrl:leftOperand": "https://eu-upcast.github.io/vocabulary/index-en.html#hasApprovalFrom",
              "odrl:operator":http: "//www.w3.org/ns/odrl/2/eq",
              "odrl:rightOperand": "The Government"
            }
          ],
          {
            "odrl:leftOperand": "http://www.w3.org/ns/odrl/2/spatial",
            "odrl:operator":http: "//www.w3.org/ns/odrl/2/eq",
            "odrl:rightOperand": "USA"
          }
        ]
      }
    ]

    "odrl:prohibition":
    [
      {
        "odrl:action": "http://www.w3.org/ns/odrl/2/transfer",
        "odrl:assignee": "https://w3id.org/dpv/owl#Entity",
        "odrl:target": "http://example.org/datasets/covid19Stats"
        "odrl:constraint":
        [
          {
            "odrl:leftOperand": "odrl:purpose",
            "odrl:operator": "http://www.w3.org/ns/odrl/2/eq",
            "odrl:rightOperand": "https://w3id.org/dpv/owl#AcademicResearch"
          }
        ]
      }
    ]


}
```

This policy is presented in a tabular form as follows

Policy Description		
Target	"http://example.org/datasets/covid19Stats"	
Rule	Permission	
	Action - Analyse	
	Refinement	**Left Operand** - hasStartTime
		Operator – Eq
		Right Operand – "Today"
	Refinement	**Left Operand** - hasFinishTime
		Operator – Eq
		Right Operand – "Tomorrow"
	Actor - Student	
	Refinement	**Left Operand** - isRepresentativeFor
		Operator – Eq
		Right Operand – "University of Harvard"
	Purpose – Academic Research	
	Refinement	**Left Operand** - hasApprovalFrom
		Operator – Eq
		Right Operand – "The Government"

	Target Refinements	
	Refinement	**Left Operand** - QUERY
		Operator – Eq
		Right Operand – "SELECT C1, C2 FROM COVID_DATA"
	Constraints	
	Constraint	**Left Operand** – Spatial (area)
		Operator – Eq
		Right Operand – "USA"
Rule	Prohibition	
	Action - Transfer	
	Actor - Entity	
	Purpose – Academic Research	
	Target Refinements	
	Constraints	

Chapter Summary/Key Takeaways

This chapter provides further technical information about how the policies defined within the editor are output as JSON-LD ODRL.

Chapter Seven: Adding the Ontology

This chapter provides an overview of how the user may load an ontology file into the policy editor.

Introduction

A brief introduction to associating the ontology files with the editor is shown in Chapter Two: The Ontology Tab.

As a reminder this is shown above the list of ontologies and illustrated in Figure 19.

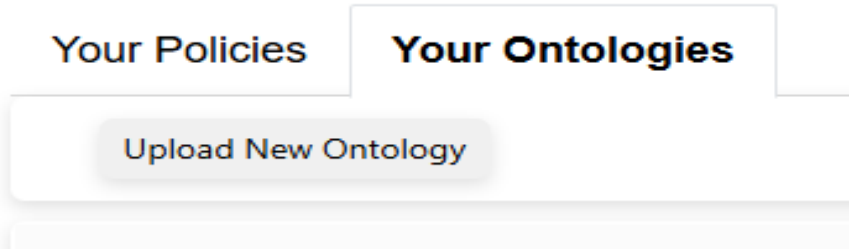

Figure 19 - General ontology page options

- If the user clicks on the 'Upload Ontology', then the user may upload a previously saved turtle or RDF format ontology file.

Example of Adding an Ontology

In this case the user has added the example ontology provided with the editor.

For illustration the text file is reproduced here

```
@prefix : <http://example.org/> .
@prefix dpv: <https://w3id.org/dpv#> .
@prefix dpv-owl: <https://w3id.org/dpv/owl#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix odrl: <http://www.w3.org/ns/odrl/2/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix profile: <http://www.w3.org/ns/dx/prof/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix role: <http://www.w3.org/ns/dx/prof/role/> .
@prefix schema: <https://schema.org/> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .

# ACTIONS
================================================================
```

```
        :CorrelationAnalysis a odrl:Action , skos:Concept ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "Correlation Analysis"@en ;
                odrl:includedIn dpv-owl:Analyse .

        :GridBasedClustering a odrl:Action , skos:Concept , dpv-owl:Processing ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "Grid-based Clustering"@en ;
                odrl:includedIn dpv-owl:Analyse .
        :hasThresholdDensity rdfs:domain :GridBasedClustering.

        :ModelBasedClustering a odrl:Action , skos:Concept , dpv-owl:Processing ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "Model-based Clustering"@en ;
                odrl:includedIn dpv-owl:Analyse .
        :basedOnModel rdfs:domain :ModelBasedClustering.

        # LEFT OPERANDS
================================================================

        :hasThresholdDensity a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "has threshold density"@en .

        :basedOnModel a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "is based on model"@en .

        :hasSponsor a odrl:LeftOperand , owl:NamedIndividual , skos:Concept ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "is sponsored by"@en .

        # PURPOSES
================================================================

        :MedicalResearch a dpv-owl:Purpose , skos:Concept ;
                rdfs:isDefinedBy <http://example.org> ;
                rdfs:label "Medical Research"@en ;
                skos:prefLabel "Medical Research"@en ;
                skos:broader dpv-owl:ScientificResearch .
        :hasSponsor rdfs:domain :MedicalResearch .

        # ACTORS
================================================================

        :PartTimeEmployee a rdfs:Class , skos:Concept ;
                rdfs:subClassOf dpv-owl:Employee ;
                rdfs:label "part-time employee"@en ;
                skos:broader dpv-owl:Employee .

        :FullTimeEmployee a rdfs:Class , skos:Concept ;
```

```
rdfs:subClassOf dpv-owl:Employee ;
rdfs:label "full-time employee"@en ;
skos:broader dpv-owl:Employee .
```

Following loading of this ontology, the user will now see this listed as one of the ontologies used (Figure 20).

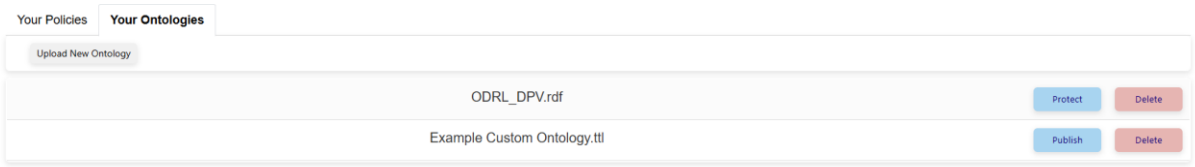

Figure 20 - Adding an ontology

Prior to loading the new ontology, the default Actor dropdown will look like this (Figure 21).

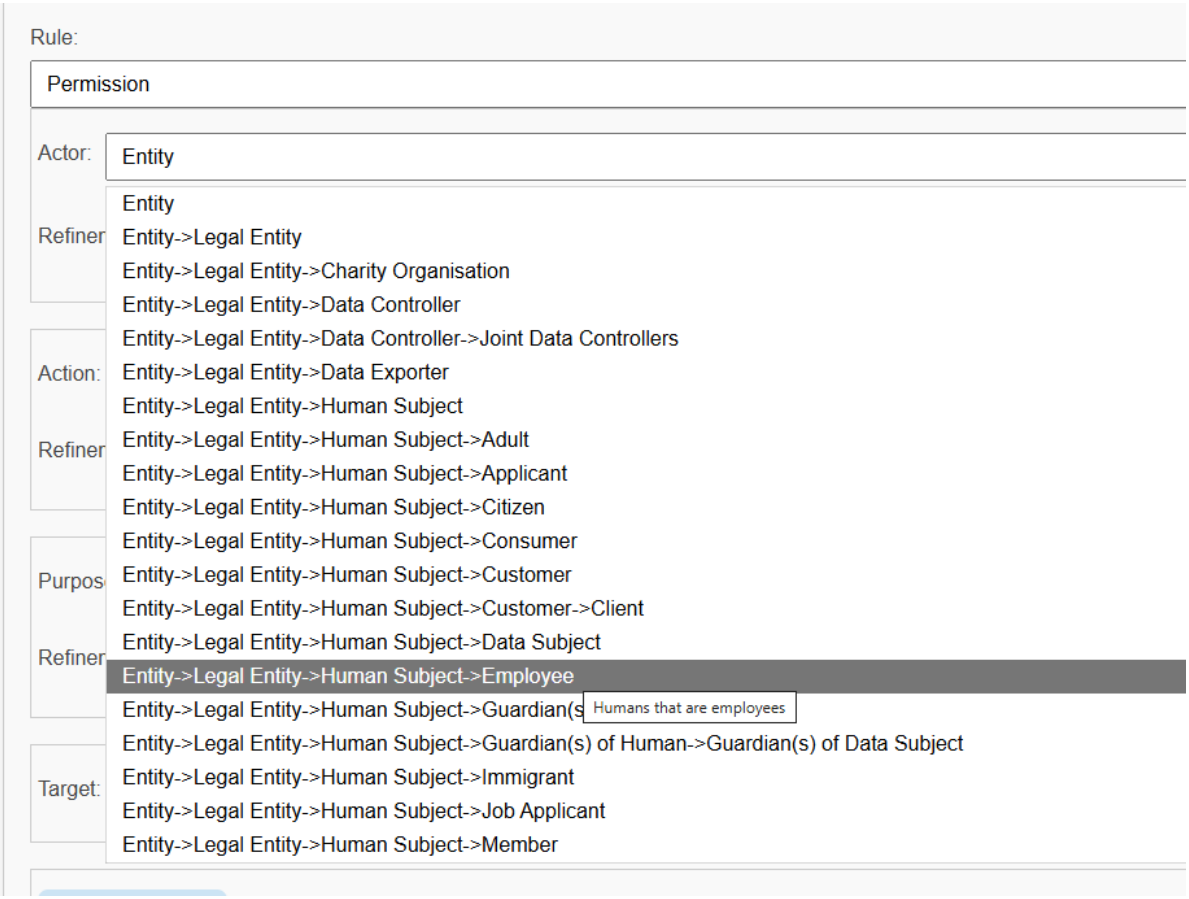

Figure 21 - Actor dropdown default ontology

Following load of the new ontology, the default Actor dropdown will look like this (Figure 22).

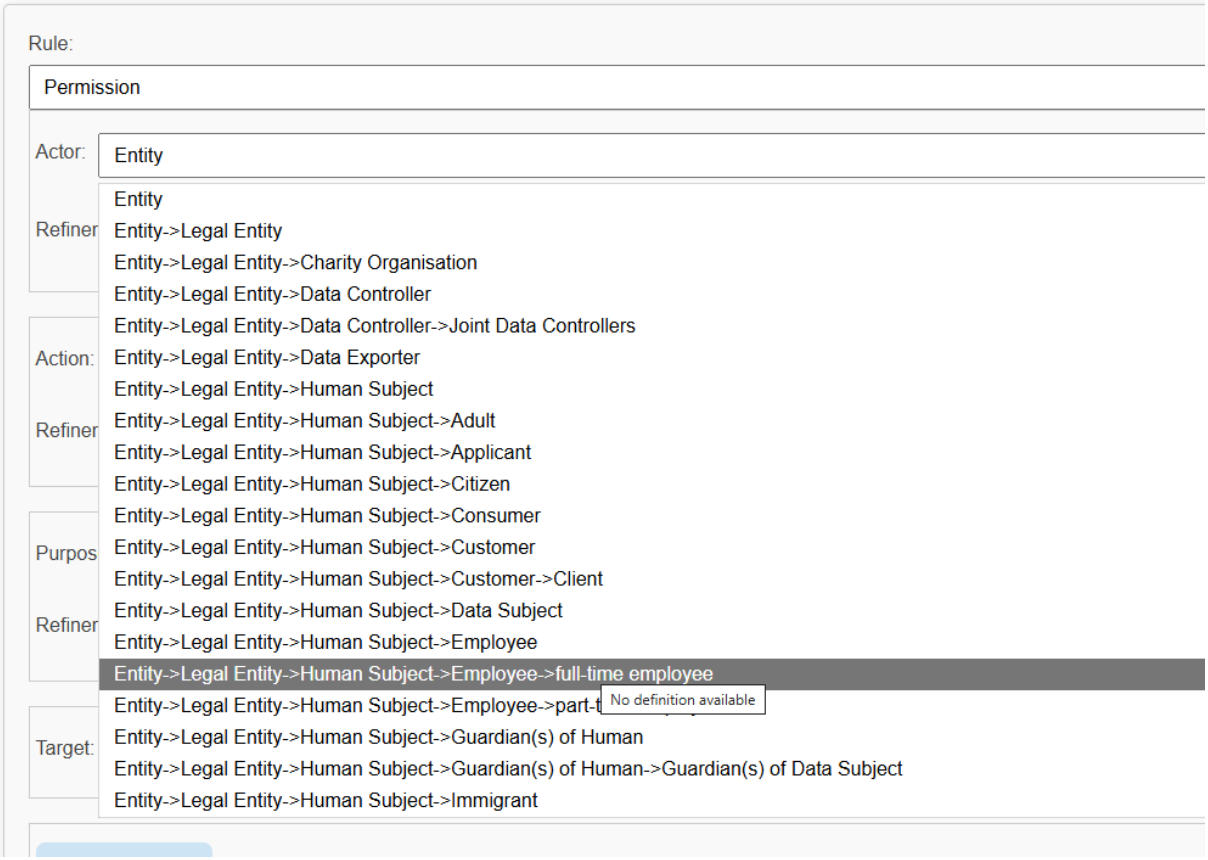

Figure 22 - Actor dropdown with user defined values

As can be seen, the user-defined values are now available for use within this field. This broad principle is true across all of Actor, Action, Purpose and associated Refinement dropdowns.

Chapter Summary/Key Takeaways

This chapter provides a brief summary of how to load ontologies and a brief demonstration of how the vocabulary available may be changed to enhance the expressivity of the policies being defined.

Section 4 – The Policy Engine

This section provides an overview of the Policy Engine key functionalities.

Here we are concerned with the operation of the Policy Evaluation page access through the ‘Evaluate Policies’ button shown in Figure 23.

Figure 23 - Policy Evaluation button

Chapter Eight: Policy Evaluation

This chapter provides a summary of the Policy Evaluation page, as illustrated in Figure 23.

Overview

This page allows the user to select a state of the world file and one or more policies (to merge) and then evaluate the state of the world against this merged set of policies.

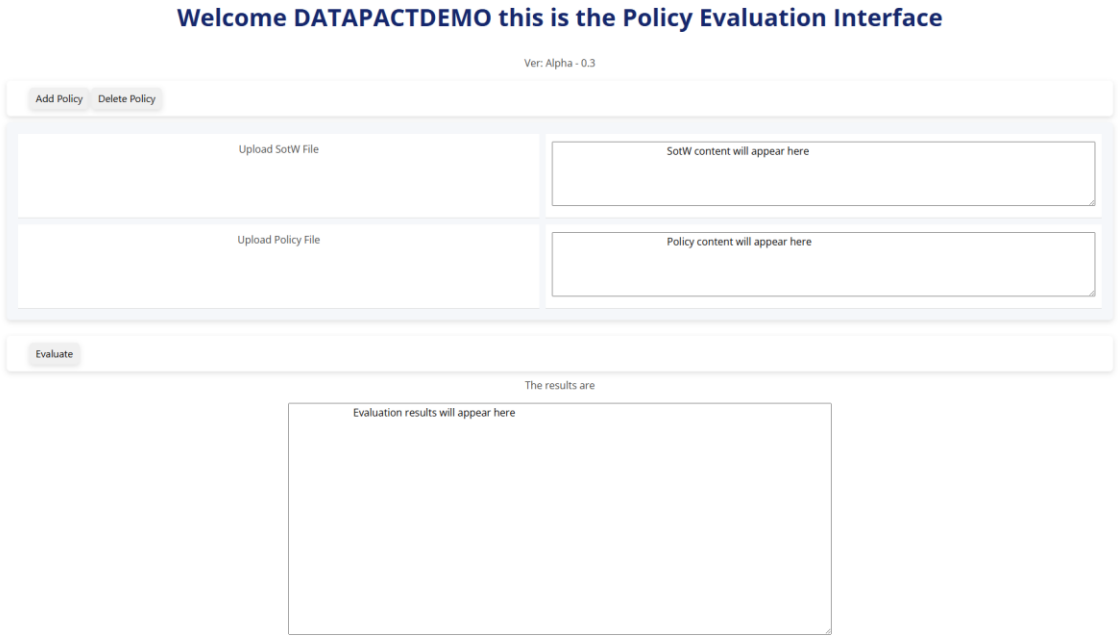

Figure 24 - The Policy Evaluation page

The user selects a state of the world file by pressing the 'Upload SotW File' button and selecting the required file. As shown in Figure 25.

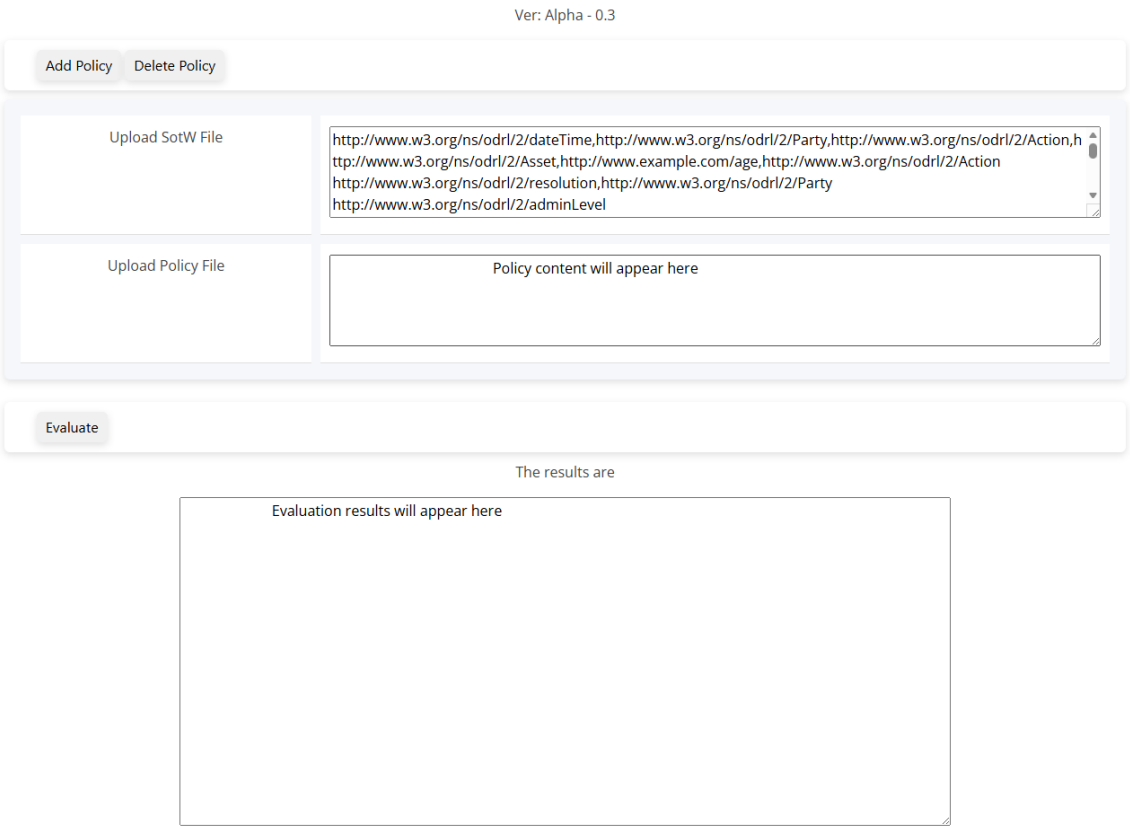

Figure 25 - SotW Upload

The user then selects a policy file by pressing the 'Upload Policy File' button and selecting the required file. As shown in Figure 26.

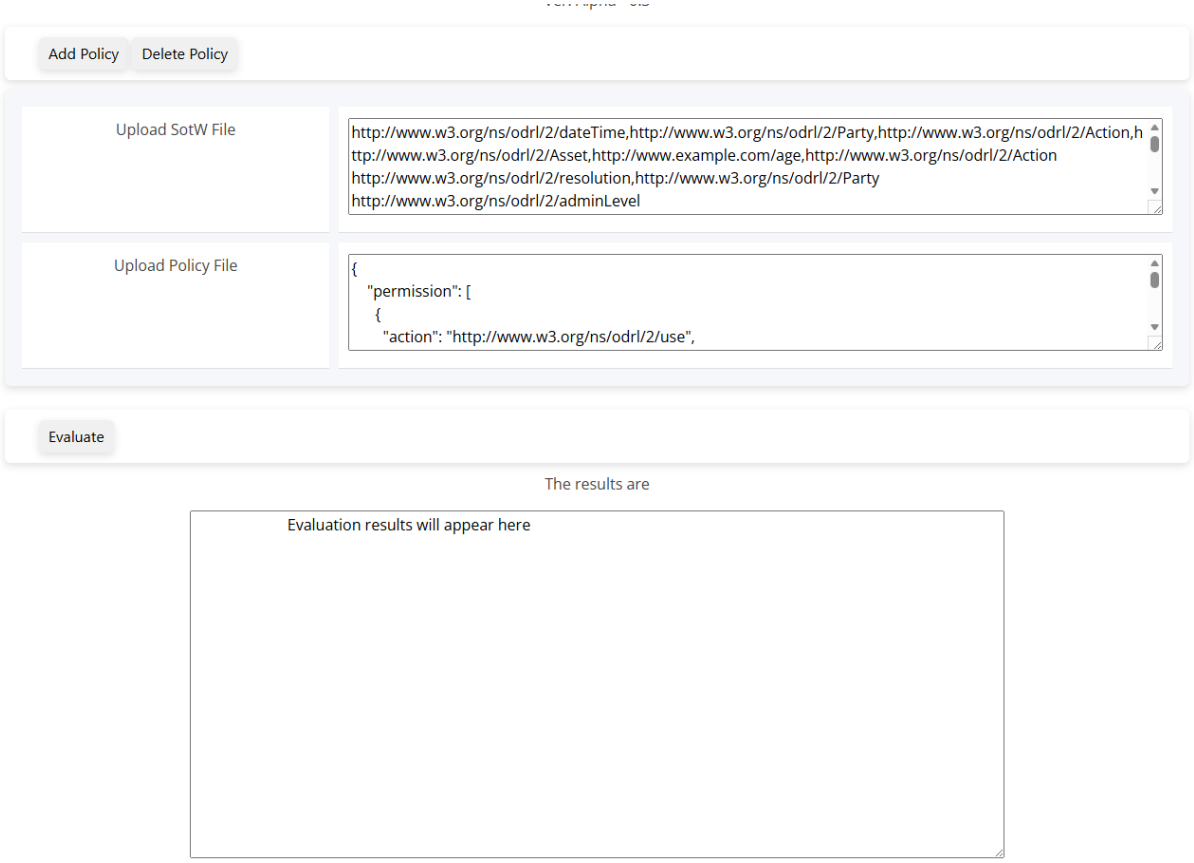

Figure 26 - Upload SotW file

The user may then evaluate the given state of the world against the given policy by pressing the 'evaluate' button. As shown in Figure 27.

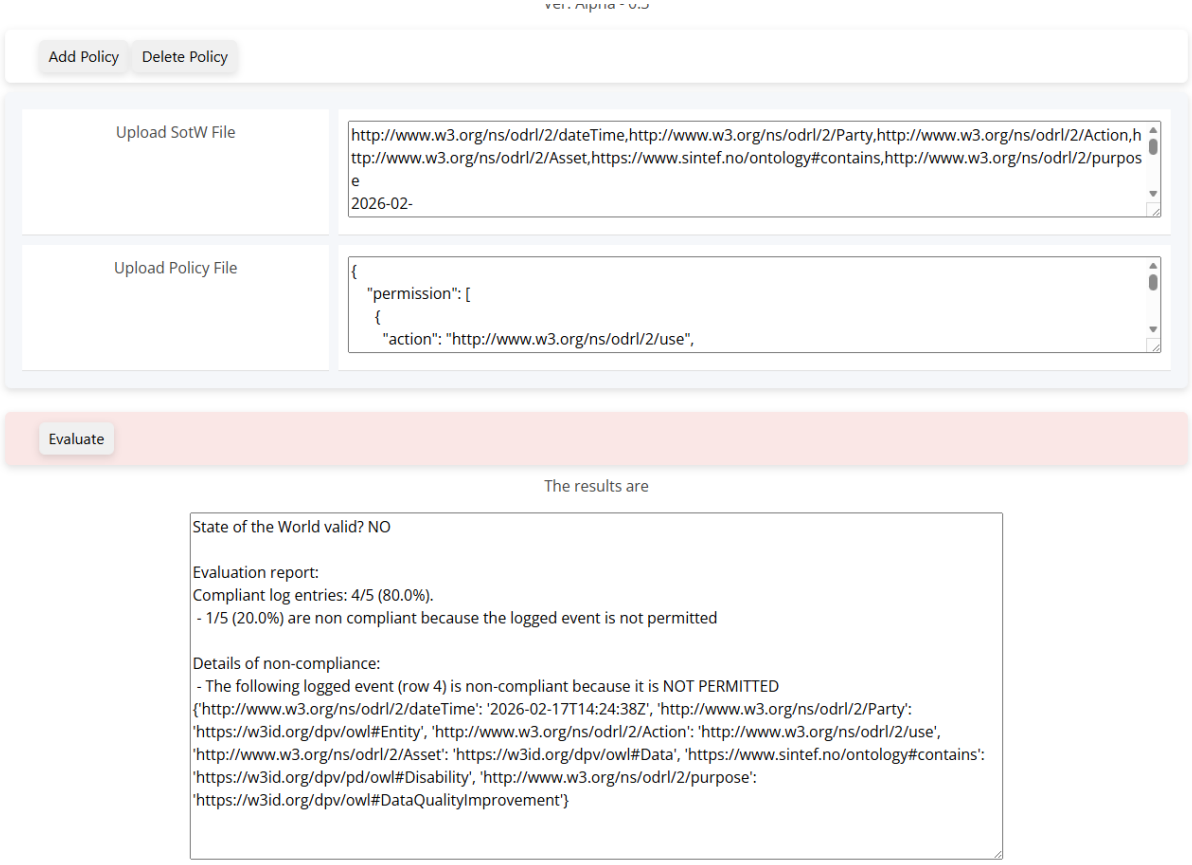

Figure 27 - Failed evaluation

Here the evaluation has failed we are told that one of the events is non-compliant. In this example we load an additional policy with the required permissions. The additional policy is loaded by pressing the ‘Add Policy’ button (pressing the ‘Delete Policy’ button will remove the last policy). This creates a new dialogue ‘row’ to allow loading of the additional policy. These two policies are merged, and the evaluation is now successful. As shown in Figure 28.

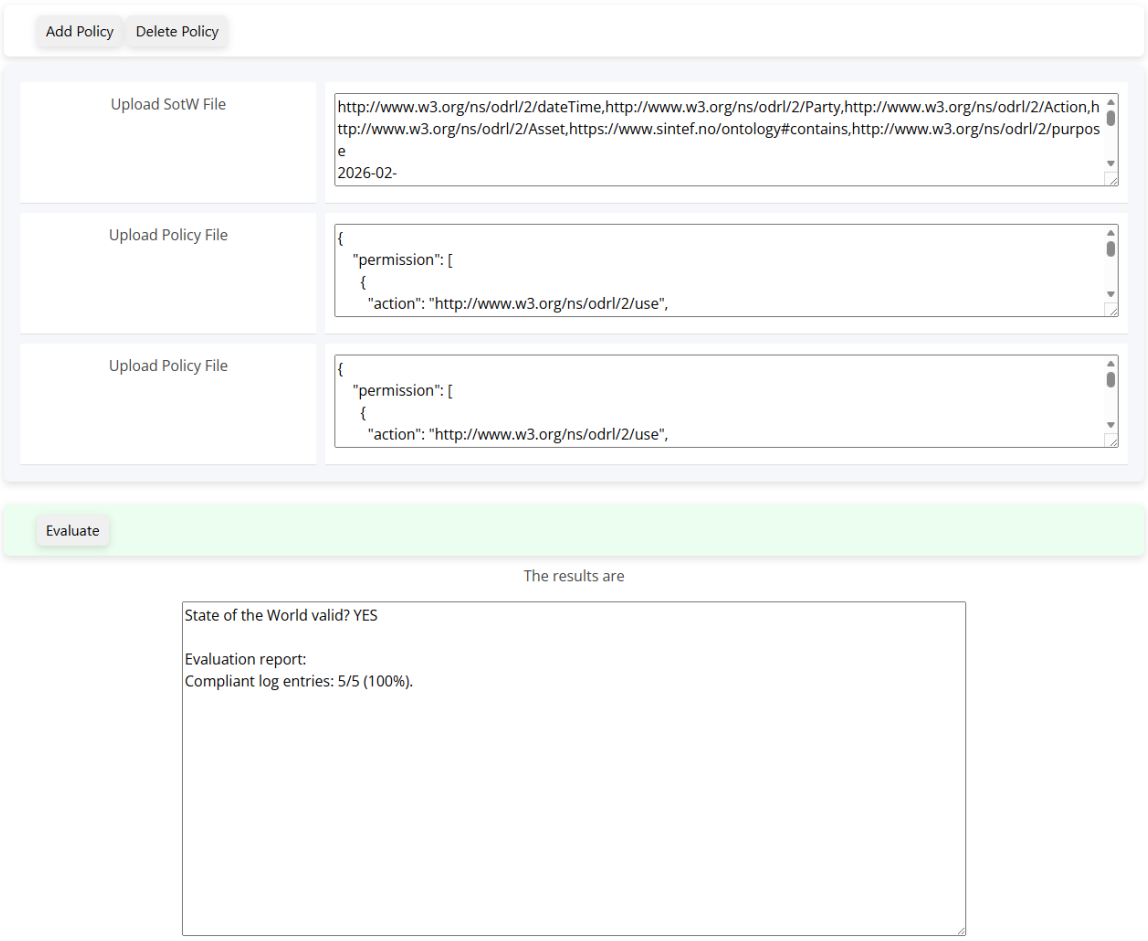

Figure 28 - Successful evaluation

Chapter Summary/Key Takeaways

This chapter provides a summary of how to access the policy evaluation functionality of the policy engine. One or more policies are merged and then a State of the World 'event stream' is evaluated against the policies.